



Interactive Knowledge Graph Querying Through Examples and Facets

Yael Amsterdamer^(✉) and Laura Gáspár

Bar-Ilan University, Ramat Gan, Israel
yael.amsterdamer@biu.ac.il

Abstract. Knowledge graphs are a highly useful form of information representation. To assist end users in understanding the contents of a given graph, multiple lines of research have proposed and studied various data exploration tools. Despite major advancements, it remains highly non-trivial to find entities of interest in a large-scale graph where the user requirements may depend on the initially unknown contents and structure of the graph. We provide in this paper a formal approach for the problem, which combines in a novel way ideas from two approaches: query-by-example and faceted search. We first provide a novel model for user interaction that includes different formal semantics for interpreting the answers. The semantics correspond to natural interpretations of feedback in faceted search. We show that for each of these semantics, any sequence of user feedback may be encoded as a SPARQL query under standard closed-world semantics. We then turn to the problem of iteratively choosing which user feedback to prompt in order to optimize the expected length of interaction. We show that depending on the probabilities of user answers, the optimal choice of question may depend on the semantics; in contrast, we show that for a natural way of estimating the probabilities, the optimal choices coincide.

1 Introduction

The widespread adoption of knowledge graphs (KGs) as means for representing information calls for effective ways to allow users to query and explore them. SPARQL, the predominant query language for RDF graphs, allows specifying complex data selection criteria. Yet, writing formal queries requires the user not only to master the query language, but also to be familiar with the contents and structure of the queried KG, and to have a crisp notion of a question to be asked over this data. Due to the difficulty of these tasks in light of the increasing scale of KGs, assisting the uninformed user in identifying *relevant parts thereof* is a crucial need.

These challenges are well known and have been extensively studied, leading to the development of dedicated *data exploration tools*. There is a wide range of approaches and technologies for this task (see [9] for a survey). A prominent approach is that of *query-by-example* (e.g., [1, 2, 4, 10, 12, 13, 16, 19, 23, 26, 30]), which aims at the “reverse-engineering” of a query from output examples. The work of [2, 4, 12, 19] has developed such solutions specifically for SPARQL.

Another prominent approach, *faceted search*, allows users to browse through criteria that may be added to a gradually forming query, typically by providing a friendly interface for criteria selection, and dynamically updating the results (e.g., [5, 6, 11, 17, 27, 28]).

Despite this great progress, the problem is far from being solved. In some cases, users may be unable to provide enough output examples (for query-by-example); and browsing the list of properties (as in faceted search) may be ineffective since properties are numerous, sparse, and not shown in context of entities and other properties. Faceted search may also lead to a “dead end” when the combination of properties does not capture the desired result [3]. Consider, for instance, a criminologist examining governmental data published in RDF, with the goal of studying properties of “interesting criminals”. Since this user is not familiar with the contents of the repository, she may struggle in finding relevant information. Presenting the user example entities and properties may help her discover, e.g., that “a criminal” in this repository is identified through `convictedOf` properties, or realize that the data contains many historic convicts while she is interested in relatively recent convictions of living people (and what properties identify this relevant data).

We develop an approach for querying a given KG for entities based on their properties without prior knowledge of the KG contents. Our approach “marries” ideas from query-by-example and faceted search. Briefly, the framework selects example entities as well as example properties thereof to show the user. The display of entities with multiple properties provides context to understand their meaning and use in the KG. The user is then asked to provide feedback for the properties, in the spirit of faceted search. Following interactive variants of query-by-example, the entities and subsets of properties are chosen to maximize the expected information gain at each step. Then, feedback at the property level rather than entity level can greatly speed up the convergence to the user intention. This paper lays formal foundations for the proposed approach, making the following contributions.

Formal Model for User Interaction. Our first contribution is a formal model for user interactions. Questions are properties of entities in the KGs and answers take the form of “yes”/“no”/“Don’t care”. The novelty lies in the interpretations of answers: KBs are typically incomplete, e.g., when they are built through automated processes that may fail to cover all real-world facts, either due to recall issues or due to incomplete data sources. In such cases an entity may in fact match the user intention (e.g., be an “interesting criminal”) even if a requested property (e.g., conviction date) is not present with respect to it. We provide four different semantics for interpreting user feedback and defining its effect on qualifying entities. We show that there is a chain of inclusion between the sets of qualifying entities according to the four semantics. Thus, the semantics may be viewed as means of balancing the precision and recall of queries, taking into account the incomplete nature of KGs.

Encoding in Standard Semantics. User answers provide an evolving specification of entity properties. We show that regardless of the semantics of user interac-

tion, the specification may be encoded as a SPARQL query under standard closed-world semantics that SPARQL engines use, and that further the resulting SPARQL query is a member of a simple SPARQL fragment. Retrieving the relevant entities then simply involves invoking a SPARQL query engine.

Choosing the Right Questions. Given a choice of semantics for user interaction, the problem is then to choose questions in a way that minimizes the set of candidate entities. We provide a natural probabilistic formulation of the problem and show that, in general, the optimal choice of question may depend on the semantics assigned to user feedback. In contrast, we show a concrete way of inferring the probabilistic distribution of anticipated future answers based on statistics of past answers, and show that in this case, the optimal question to ask no longer depends on the semantics of user feedback.

2 Model

We next provide a model for KGs and modes user interaction.

2.1 Knowledge Graphs

We use here a simple KG model in the spirit of languages such as RDF and OWL, abstracting away details that are not necessary for our setting. Let $\mathcal{E}$ and $\mathcal{P}$ be sets of element ids and property ids respectively, standing for entities/concepts from the knowledge domain, and their properties. We further allow defining literal values from Σ^* assuming $\Sigma^* \cap \mathcal{E} = \Sigma^* \cap \mathcal{P} = \emptyset$.

Definition 1 (Facts and knowledge graphs). A fact over $\mathcal{E}$, $\mathcal{P}$, Σ^* is a triple of the form $\{\text{subject property object}\}$ where $\text{subject} \in \mathcal{E}$, $\text{property} \in \mathcal{P}$ and $\text{object} \in \mathcal{E} \cup \Sigma^*$. A KG is a set of facts.

Example 1. The KG could include facts $\{\text{Saddam.Hussein type Leader}\}$ and $\{\text{Saddam.Hussein birthDate "1937-04-28"}\}$. In this case, $\text{Saddam.Hussein, Leader} \in \mathcal{E}$, $\text{type, birthDate} \in \mathcal{P}$ and $\text{"1937-04-28"} \in \Sigma^*$.

A KG can be also viewed as a labelled directed graph with parallel edges and self-edges, where the vertices are elements in $\mathcal{E}$ or Σ^* , and every directed edge $(\text{subject}, \text{object})$ is labelled by some property $\in \mathcal{P}$. We interpret the meaning of a KG under an *open world assumption*: facts in the KG are asserted to be true and facts not in it may be true or false. This allows incompleteness, which typically holds in practice, as explained above.

2.2 Interaction Model

Let u be a user who seeks entities in the KG $\mathcal{G}$. Denote by E_u the set of all entities that are acceptable by u . To identify E_u , we ask u questions that can be interpreted as “Should entities in E_u have property p with object o ?” e.g., “Should entities in E_u have the property **type** with object **Leader**?” Formally,

```

1 SELECT DISTINCT $e
2 WHERE {$e type Person. $e convictedOf []}
3     MINUS {$e deathPlace []}
4     MINUS {$e convictedOf WarCrimes}.)}

```

Fig. 1. Example selection query Q_{ex} .

Definition 2 (Question and answer model). A user question in our framework is denoted by $q_{p,o}$ where $p \in \mathcal{P}$ and $o \in \mathcal{E} \cup \Sigma^* \cup \{\square\}$ ($\square$ stands for an undistinguished variable, allowing any value to be assigned to it). In response a user u chooses an answer $a \in \{\square p(o), \square \neg p(o), \diamond p(o)\}$, where for $o \neq \square$, $\square p(o)$ (resp., $\square \neg p(o)$) implies that for every $e \in E_u$, the fact $\{e \ p \ o\}$ is true (resp., is false); $\diamond p(o)$ implies that this fact may or may not hold for any $e \in E_u$ (“don’t care”). If $o = \square$, $\square p(\square)$ (resp., $\square \neg p(\square)$) implies that for every $e \in E_u$ there is some (no) value $o' \in \mathcal{E} \cup \Sigma^*$ such that the fact $\{e \ p \ o'\}$ is true (resp., is false).

We next recall the syntax of a simple fragment of SPARQL, which we refer to as **sSPARQL**, and which will be used to encode user answers.

Definition 3. A fact pattern is a triple $\$e \ p \ o$ where $\$e$ is a variable (fixed for a given query) and $p \in \mathcal{P}$ and $o \in \mathcal{E} \cup \Sigma^* \cup \{\square\}$. A fact $\{s \ \text{pred} \ \text{obj}\}$ matches the pattern if $p = \text{pred}$, if either $o = \text{obj}$ or $o = \square$. s is assigned to $\$e$. We say that an entity s matches a pattern P with respect to a KG $\mathcal{G}$ if there exists a fact $\{s \ \text{pred} \ \text{obj}\} \in \mathcal{G}$ that matches P .

An **sSPARQL** query is composed of a **SELECT** and **WHERE** clauses, as well as an optional **MINUS** operator. The **WHERE** clause consists of groups of fact patterns (in curly brackets), which may be nested in **MINUS** operators. Queries in our fragment always contain the clause **SELECT DISTINCT \$e**, which means the result of a query Q over a KG $\mathcal{G}$, denoted $Q(\mathcal{G})$, includes every distinct assignment to $\$e$ that is consistent with the query contents (whose semantics are defined below).

Example 2. Figure 1 illustrates a query selecting all human convicts of who are still alive and have not been convicted of war crimes. This is done by selecting entities (assignments to $\$e$) that e.g., do not appear in a fact with a **deathPlace** predicate (with any object), etc.

Let $\{\langle q_{p_1,o_1}, a_1 \rangle, \dots, \langle q_{p_n,o_n}, a_n \rangle\}$ be a set of questions and respective answers after n interaction steps. We encode the answers in a query Q_n , where for each i , $a_i = \square p_i(o_i)$ is encoded by $\$e \ p_i \ o_i$ and $a_i = \square \neg p_i(o_i)$ is encoded by **MINUS** $\{\$e \ p_i \ o_i\}$. Q_n does not encode $\diamond p(o)$, as it has no effect on the selected entities (but we record the answer, to avoid repeating a question).

Example 3. The query in Fig. 1 is an encoding of the answer sequence $\diamond \text{gender}(\text{male}), \square \neg \text{deathPlace}(\text{Any}), \square \text{convictedOf}(\text{Any}), \square \neg \text{convictedOf}(\text{WarCrimes}), \square \text{type}(\text{Person})$. Note that the order of answers is not important, and that the first, “Don’t care” answer is not encoded.

2.3 Multiple Semantics for User Interactions

As explained in the Introduction, there are multiple reasonable semantics that could be assigned to the user feedback, and thereby to the **sSPARQL** query corresponding to it. We can also view the different semantics as different ways of balancing the precision and recall of queries over an incomplete KG with respect to a (complete) ground truth. We next overview these semantics.

Closed-World Semantics. Given a query over a KG $\mathcal{G}$, typical SPARQL engines interpret it as follows: an assignment s to $\$e$ is consistent with a pattern group if for each pattern p in the group s matches p with respect to $\mathcal{G}$ / (This also holds if a matching fact could be *inferred* from $\mathcal{G}$'s contents, using logical inference rules as in RDF Schema and OWL.) **MINUS** has the usual meaning of difference. This semantics is closed-world in the sense that we regard missing facts as false.

Open-World Semantics. Closed-world semantics strictly excludes entities with missing information. An alternative semantics, which complies with the open-world assumption, ignores fact patterns in a “positive” context, i.e., not nested in negation. We thus cannot prune entities for which a matching fact does not occur in the graph; however, **MINUS** has the same semantics as in Closed-world.

Step-Wise Semantics. Closed world semantics may miss valuable results due to missing facts, while Open-world semantics misses important characterization of entities via positive fact patterns. As an intermediate solution, step-wise semantics has the same semantics as closed-world, as long as query result set is not empty. If it is empty, the query results contain all the entities that match a *maximal subset* of the fact patterns in positive context.

Weighted Semantics. Finally, we consider a semantics that accounts for *which* positive fact patterns are matched by each selected entity, and the importance of each constraint. For that, we associate with each fact pattern group G an importance weight $w(G)$. Let $G(\mathcal{G})$ denote the set of entities that match the pattern group under Closed-world Semantics, and let $\{G_1, \dots, G_m\}$ denote the maximal subsets of positive constraints used in Step-wise semantics. We will then return the entities that match one of $\arg \max_{G_i} w(G_i)$ as well as the **MINUS** constraints. An example weight function, in the spirit of information theory is $w(G) = -\log \left(\frac{|G(\mathcal{G})|}{|\mathcal{G}|} \right)$, i.e., inverse to the number of entities matching G .

We next exemplify the different semantics.

Example 4. Reconsider the query Q_{ex} in Fig. 1, and now assume a small KG $\mathcal{G}_{\text{ex}}$ that contains the following facts: **{Saddam.Hussein deathPlace Baghdad. Saddam.Hussein type Person. Silvio.Berlusconi convictedOf Fraud. Angela.Merkel type Person.}** The entity of **Saddam.Hussein** matches the third pattern and hence does not match the first **MINUS** constraint, and will not appear in the query result under any of our semantics.

Under Closed-world semantics, $Q_{\text{ex}}(\mathcal{G}_{\text{ex}}) = \emptyset$, as each entity does not match some constraint: `Silvio_Berlusconi` and `Angela_Merkel` do not match the first pattern and the second patterns respectively.

Under Open-world semantics, we ignore the positive constraints and will return both of these entities.

Under Step-wise semantics, we have two maximal subsets of the patterns in positive context that match some entity: `{ $e type Person }` and `{ $e convicted of [] }`, matched by `Silvio_Berlusconi` and `Angela_Merkel` respectively. Hence in this case, Step-wise semantics yields the same output as Open-world.

Finally, assuming a weight function inverse to number of matching entities, the second pattern will have a higher weight (having only one matching person, as opposed to two matching the first pattern). This means we will return the results adhering to the maximal subset of patterns containing only the second pattern, and adhering to the MINUS constraints - only `Silvio_Berlusconi`. This makes sense if, intuitively, the omission of `Silvio_Berlusconi type Person` is more likely than the omission of `Angela_Merkel convictedOf o` for any `o`. $\square$

Proposition 1. *For any Q and $\mathcal{G}$, it holds that $Q_{\text{Closed-world}}(\mathcal{G}) \subseteq Q_{\text{Weighted}}(\mathcal{G}) \subseteq Q_{\text{Step-wise}}(\mathcal{G}) \subseteq Q_{\text{Open-world}}(\mathcal{G})$*

Assume we start with the entire domain $\mathcal{E}$ as the set of candidate entities. We may also show that under Closed-world and Open-world semantics, a sequence of questions and answers yields queries whose set of answers monotonically decreases. This matches the intuition that query answers are equivalent to constraints that serve to narrow down the set of candidate entities.

Proposition 2. *Let $\langle q_{p_1, o_1}, a_1 \rangle, \dots, \langle q_{p_n, o_n}, a_n \rangle$ be a sequence of questions and answers. For every i , $1 \leq i < n$, and any input graph $\mathcal{G}$, $Q_{\text{sem}}^{i+1}(\mathcal{G}) \subseteq Q_{\text{sem}}^i(\mathcal{G})$, where Q^i is the query encoding of $\{\langle q_{p_1, o_1}, a_1 \rangle, \dots, \langle q_{p_i, o_i}, a_i \rangle\}$, and $Q_{\text{sem}}^i(\mathcal{G})$ is its evaluation on $\mathcal{G}$ under semantics *sem* which is one of Closed-world, Open-world.*

This monotonicity property does not hold for Step-wise and Weighted semantics, since as more constraints are added, new fact pattern subsets that select additional entities may be formed.

Encoding in Closed-World Semantics. Since standard SPARQL engines evaluate queries under Closed-world semantics, it may be useful to encode queries under other semantics by their Closed-world equivalents. Indeed, we may show:

Proposition 3. *For every sSPARQL query Q and a KG G , there exist sSPARQL queries Q^1, Q^2, Q^3 such that*

- $Q_{\text{Step-wise}}(\mathcal{G}) = Q_{\text{Closed-World}}^1(\mathcal{G})$
- $Q_{\text{Weighted}}(\mathcal{G}) = Q_{\text{Closed-World}}^2(\mathcal{G})$
- $Q_{\text{Open-World}}(\mathcal{G}) = Q_{\text{Closed-World}}^3(\mathcal{G})$.

Proof. We next provide the encoding for each semantics.

- *Open-world to Closed-world.* Given a query Q under Open-world semantics, a query Q' such that $Q_{\text{Open-world}} \equiv Q'_{\text{Closed-world}}$ can be obtained by omitting the positive constraints from Q .
- *Step-wise to Closed-world.* Given a query Q under Step-wise semantics, a query Q' such that $Q_{\text{Step-wise}}(\mathcal{G}) = Q'_{\text{Closed-world}}(\mathcal{G})$ for a given KG $\mathcal{G}$ can be obtained by using a UNION operation, which operates over two fact pattern groups and is supported by standard SPARQL engines under the standard (Closed-world) semantics: define a group for each maximal subset of the fact patterns, and replace the positive part of the query by this union.
- Given a query Q under Weighted Semantics, a query Q' s.t. $Q_{\text{Step-wise}}(\mathcal{G}) = Q'_{\text{Closed-world}}(\mathcal{G})$ for a given KG $\mathcal{G}$ and weight function w can be obtained similarly to Step-wise semantics, except that we take the union of the groups with the maximal weight only.

Note that for Step-wise and Weighted semantics, the encoding depends on the input graph, since these semantics depend on the (non-)emptiness of queries. The encoding of the latter further depends on the weight function. To bind $\$e$ when the positive part of the query is empty, we have added the fact pattern $\$e \square \square$. When a union consists of only one group, we replace it by that group.

3 Optimizing the Interaction

We next develop a framework for iteratively choosing the questions to users, namely combinations of property and object that should/not hold for the sought entities, conditional on the semantics of previous answers. In this framework, we focus on the following scenario.

- Questions of the form $q_{p,o}$ are the only means of interacting with users, as opposed to e.g., allowing to users to type input such as search strings.
- For a given query semantics sem , there exists a query Q in our fragment such that $Q_{\text{sem}}(\mathcal{G}) = E_u$.
- Users always give answers that are accurate to their needs by Definition 2, as opposed to users who may make mistakes.
- The process can halt as soon as an entity $e \in E_u$ is presented to the user, as opposed to only when the intended query Q is discovered.

These assumptions follow a “clean” model of exploration focusing on the type of questions that we consider (Assumptions (a)–(c)), and allowing for fast convergence (Assumption (d)). Under these assumptions, we consider the choice of questions that would maximize the expected number of eliminated entities, using a probabilistic model for answers.

Definition 4 (Problem definition). Denote by $Pr_{p,o}(\square)$ and $Pr_{p,o}(\square \neg)$ the probability of “must” and “must not”, respectively, for a specific question $q_{p,o}$ at a certain point of the interaction. The probability of “don’t care” is then $Pr_{p,o}(\diamond) = 1 - Pr_{p,o}(\square) - Pr_{p,o}(\square \neg)$. Let E_φ and $E_{\varphi'}$ denote, respectively, the

output of the current query φ , encoding the user answers thus far, and the output of the next query φ' . Let $E_{p,o} \subseteq E_\varphi$ be the set of currently relevant entities e (i.e., in the output of the current φ) with the property $\{e \ p \ o\}$. The expected number of eliminated entities for a question $q_{p,o}$, under Closed-world semantics is then

$$\mathbb{E}[|E_\varphi - E_{\varphi'}|] = \Pr_{p,o}(\Box) |E_\varphi - E_{p,o}| + \Pr_{p,o}(\Box \neg) |E_{p,o}|$$

For Open-world semantics we can replace $|E_\varphi - E_{p,o}|$ by 0. We seek the question $q_{p,o}$ that maximizes this quantity.

3.1 Assuming Known Probabilities

Problem Definition 4 gives rise to a simple greedy algorithm that computes the expected probability for each combination of property and object, and then poses the question that maximizes this value. We next describe two optimizations to this algorithm. First, we observe that only questions in $\{q_{p,o} \mid \{s, p, o\} \in \mathcal{G} \wedge s \in E_\varphi\} \cup \{q_{p,\Box} \mid \exists o \{s, p, o\} \in \mathcal{G} \wedge s \in E_\varphi\}$, i.e., questions on facts about candidate entities, may lead to entity elimination greater than 0. We may thus only consider questions in this set. Second, to compute the formula we need to compute $E_{p,o}$ for each question in the above mentioned set. To do so in a comparatively efficient way, we can do a single pass over all the facts $\{s, p, o\}$ such that $s \in E_\varphi$, and with each such fact increment the counter for $E_{p,o}$ and $E_{p,\Box}$, ignoring questions already asked. The set of such facts may be retrieved by adding a pattern $\$e \$p \$o$ to φ and executing a SPARQL query that returns the values of these three variables. The number of eliminated entities may differ between different semantics. For example, if the probability to a “must” answer is high, Closed-world semantics will prefer a question corresponding to a pattern that matches few entities. In contrast, Open-world semantics only eliminates entities given a negative answer, and will thus prioritize patterns that match many entities.

3.2 Estimating the Probabilities

We next consider the estimation of $\Pr_{p,o}(\Box)$ and $\Pr_{p,o}(\Box \neg)$ at each step of the interaction, assuming no external information (e.g. statistics for past user interaction). As preliminary results, we focus on the case where the user is interested in finding a single entity, with each entity in E_φ having an a-priori equal probability of being the chosen one. Let the general probability of “don’t care” be $\Pr_\Diamond$, then $\Pr_{p,o}(\Box) = (1 - \Pr_\Diamond) \frac{|E_{p,o}|}{|E_\varphi|}$, and similarly $\Pr_{p,o}(\Box \neg) = (1 - \Pr_\Diamond) \frac{|E_\varphi - E_{p,o}|}{|E_\varphi|}$. By substituting these expressions in the expectation formula above, we get that to maximize the expected number of eliminated entities with a single question $q_{p,o}$ it suffices to choose one that maximizes $|E_{p,o}| |E_\varphi - E_{p,o}|$.

To extend this estimation to sets of properties, one can define the probability of a combination of answers analogously to a single answer. A technical difficulty here, beyond the increased computational complexity, is that the $\Pr_\Diamond$ factor no longer cancels out, as the number of “don’t care” answers can vary. A possible simplification adopted by our current preliminary prototype is assuming that

properties are roughly independent, namely, for any p, o, p', o' the fraction of entities in $\frac{|E_{p,o}|}{|E_\varphi|}$ resembles $\frac{|E_{p,o} - E_{p',o'}|}{|E_\varphi - E_{p',o'}|}$, and thus it suffices to choose the k questions $q_{p,o}$ with highest $|E_{p,o}| |E_\varphi - E_{p,o}|$ values.

For open-world semantics, the first term is equal to 0, but since the second term is proportional to $|E_{p,o}| |E_\varphi - E_{p,o}|$, it turns out that for this choice of probabilities, the same questions are selected for Closed-world and Open-world semantics, even though the entities they eliminate are different.

4 Related Work

The challenges faced by users interacting with huge Knowledge Bases are well known and there is a vast body of research on knowledge graph exploration [9]. For entity search, different techniques have been proposed for KB keyword search [14, 15, 22, 29]. For query formation, one solution is using *query-by-example* to infer queries from positive/negative output examples. There is a broad line of work [1, 10, 13, 16, 26, 30] on query by example in relational databases, but these techniques are not effective for properties of KBs such as sparsity, heterogeneity and incompleteness. Existing query-by-example frameworks for knowledge graphs (e.g., [2, 4, 12, 19]) require the user to provide seed examples. *Faceted Search* is a popular technique for refining search results by proposing further constraints (e.g., [7, 8, 18, 20, 21, 24, 25, 27, 28]). Work on faceted search in knowledge graphs (e.g., [6, 17, 27]) focuses on facet computation, hierarchical browsing and visualization. Given a formal query, one may use similarity search [31] to find similar queries. To our knowledge, no previous work in this context has studied the alleviation of the exploration problem by combining query-by-example with principles of faceted search.

Acknowledgements. This work was partly funded by the Israel Science Foundation (grant No. 2015/21) and by the Israel Ministry of Science and Technology.

References

1. Abouzied, A., Angluin, D., Papadimitriou, C.H., Hellerstein, J.M., Silberschatz, A.: Learning and verifying quantified boolean queries by example. In: PODS (2013)
2. Abramovitz, E., Deutch, D., Gilad, A.: Interactive inference of SPARQL queries using provenance. In: ICDE (2018)
3. Amsterdamer, Y., Callen, Y.: Provenance-based SPARQL query formulation. In: Strauss, C., Cuzzocrea, A., Kotsis, G., Tjoa, A.M., Khalil, I. (eds.) Database and Expert Systems Applications: 33rd International Conference, DEXA 2022, Vienna, Austria, August 22–24, 2022, Proceedings, Part I, pp. 116–129. Springer, Cham (2022). https://doi.org/10.1007/978-3-031-12423-5_9
4. Arenas, M., Diaz, G.I., Kostylev, E.V.: Reverse engineering SPARQL queries. In: WWW (2016)

5. Arenas, M., Grau, B.C., Kharlamov, E., Marciuska, S., Zheleznyakov, D.: Faceted search over ontology-enhanced RDF data. In: CIKM (2014)
6. Arenas, M., Cuenca Grau, B., Kharlamov, E., Marciuška, Š., Zheleznyakov, D.: Faceted search over RDF-based knowledge graphs. *J. Web Semant.* **37–38**, 55–74 (2016). <https://doi.org/10.1016/j.websem.2015.12.002>
7. Atzori, M., Zaniolo, C.: Swipe: searching Wikipedia by example. In: WWW (2012)
8. Auer, S., Lehmann, J.: What have Innsbruck and Leipzig in common? Extracting semantics from wiki content (2007)
9. Bikakis, N., Sellis, T.K.: Exploration and visualization in the web of big linked data: a survey of the state of the art. In: LWDMM (2016)
10. Bonifati, A., Ciucanu, R., Staworko, S.: Interactive inference of join queries. In: EDBT (2014)
11. Dukka, W., Ipeirotis, P.G., Wood, K.R.: Automatic construction of multifaceted browsing interfaces. In: CIKM (2005)
12. Diaz, G., Arenas, M., Benedikt, M.: SPARQLByE: querying RDF data by example. *Proc. VLDB Endow.* **9**(13), 1533–1536 (2016)
13. Dimitriadou, K., Papaemmanouil, O., Diao, Y.: Explore-by-example: an automatic query steering framework for interactive data exploration. In: SIGMOD (2014)
14. Elbassuoni, S., Blanco, R.: Keyword search over RDF graphs. In: CIKM (2011)
15. Elbassuoni, S., Ramanath, M., Schenkel, R., Weikum, G.: Searching RDF graphs with SPARQL and keywords. *IEEE Data Eng. Bull.* **33**(1), 16–24 (2010)
16. Fariha, A., Meliou, A.: Example-driven query intent discovery: abductive reasoning using semantic similarity. *Proc. VLDB Endow.* **12**(11), 1262–1275 (2019)
17. Fuenmayor, L., Collarana, D., Lohmann, S., Auer, S.: FaRBIE: a faceted reactive browsing interface for multi RDF knowledge graph exploration. In: ISWC (2017)
18. Hearst, M.A.: Clustering versus faceted categories for information exploration. *Commun. ACM* **49**(4), 59–61 (2006)
19. Jayaram, N., Khan, A., Li, C., Yan, X., Elmasri, R.: Querying knowledge graphs by example entity tuples. In: ICDE (2016)
20. Kamath, S., Manjunath, D., Mazumdar, R.: On distributed function computation in structure-free random wireless networks. *IEEE Trans. Inf. Theor.* **60**(1), 432–442 (2014)
21. Koren, J., Zhang, Y., Liu, X.: Personalized interactive faceted search. In: WWW (2008)
22. Le, W., Li, F., Kementsietsidis, A., Duan, S.: Scalable keyword search on large RDF data. *IEEE Trans. Knowl. Data Eng.* **26**(11), 2774–2788 (2014)
23. Li, Y., Yang, H., Jagadish, H.V.: NaLIX: an interactive natural language interface for querying XML. In: SIGMOD (2005)
24. Mass, Y., Ramanath, M., Sagiv, Y., Weikum, G.: IQ: the case for iterative querying for knowledge. In: CIDR (2011)
25. Mass, Y., Sagiv, Y.: Knowledge management for keyword search over data graphs. In: CIKM (2014)
26. Psallidas, F., Ding, B., Chakrabarti, K., Chaudhuri, S.: S4: top-k spreadsheet-style search for query discovery. In: SIGMOD (2015)
27. Qarabagi, B., Riedewald, M.: User-driven refinement of imprecise queries. In: ICDE (2014)
28. Roy, S.B., Wang, H., Das, G., Nambiar, U., Mohania, M.K.: Minimum-effort driven dynamic faceted search in structured databases. In: CIKM (2008)
29. Tran, T., Wang, H., Rudolph, S., Cimiano, P.: Top-k exploration of query candidates for efficient keyword search on graph-shaped (RDF) data. In: ICDE (2009)

30. Weiss, Y.Y., Cohen, S.: Reverse engineering SPJ-queries from examples. In: Sallinger, E., den Bussche, J.V., Geerts, F. (eds.) PODS (2017)
31. Zheng, W., Zou, L., Peng, W., Yan, X., Song, S., Zhao, D.: Semantic SPARQL similarity search over RDF knowledge graphs. *Proc. VLDB Endow.* **9**(11), 840–851 (2016)